

通过Cline使用智能体：代码案例

讲师：尚硅谷宋红康（江湖人称：康师傅）

官网：www.atguigu.com

douyin账号：是康师傅呀(尚硅谷)

Cline是VS Code的插件，可以通过调用LLM的API实现自动化开发，可被视为用于开发智能体。

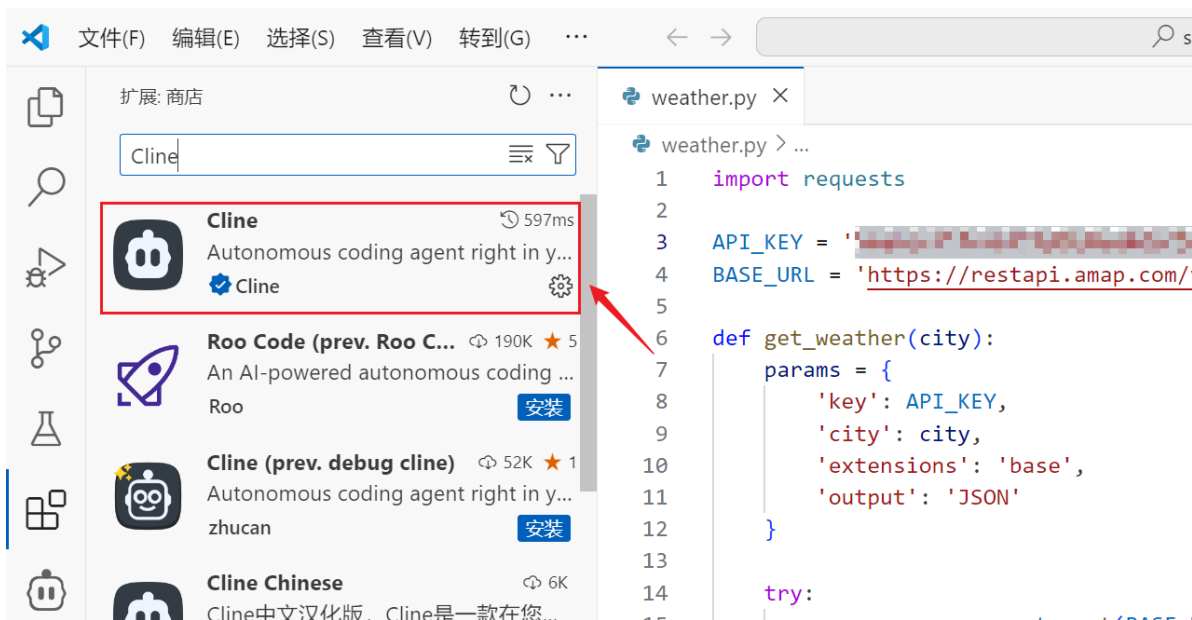
它是一个github上的开源项目，仓库地址如下

<https://github.com/cline/cline>

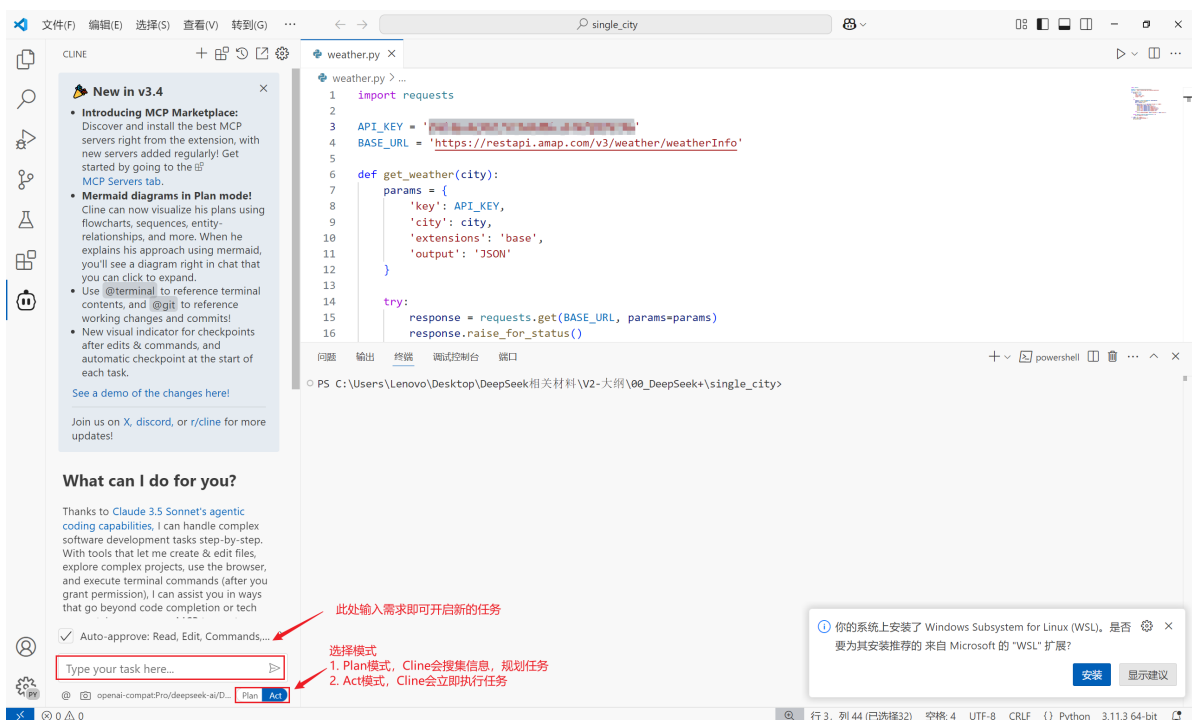
1、VS Code配置

通过Cline插件将DeepSeek集成到VSCode。

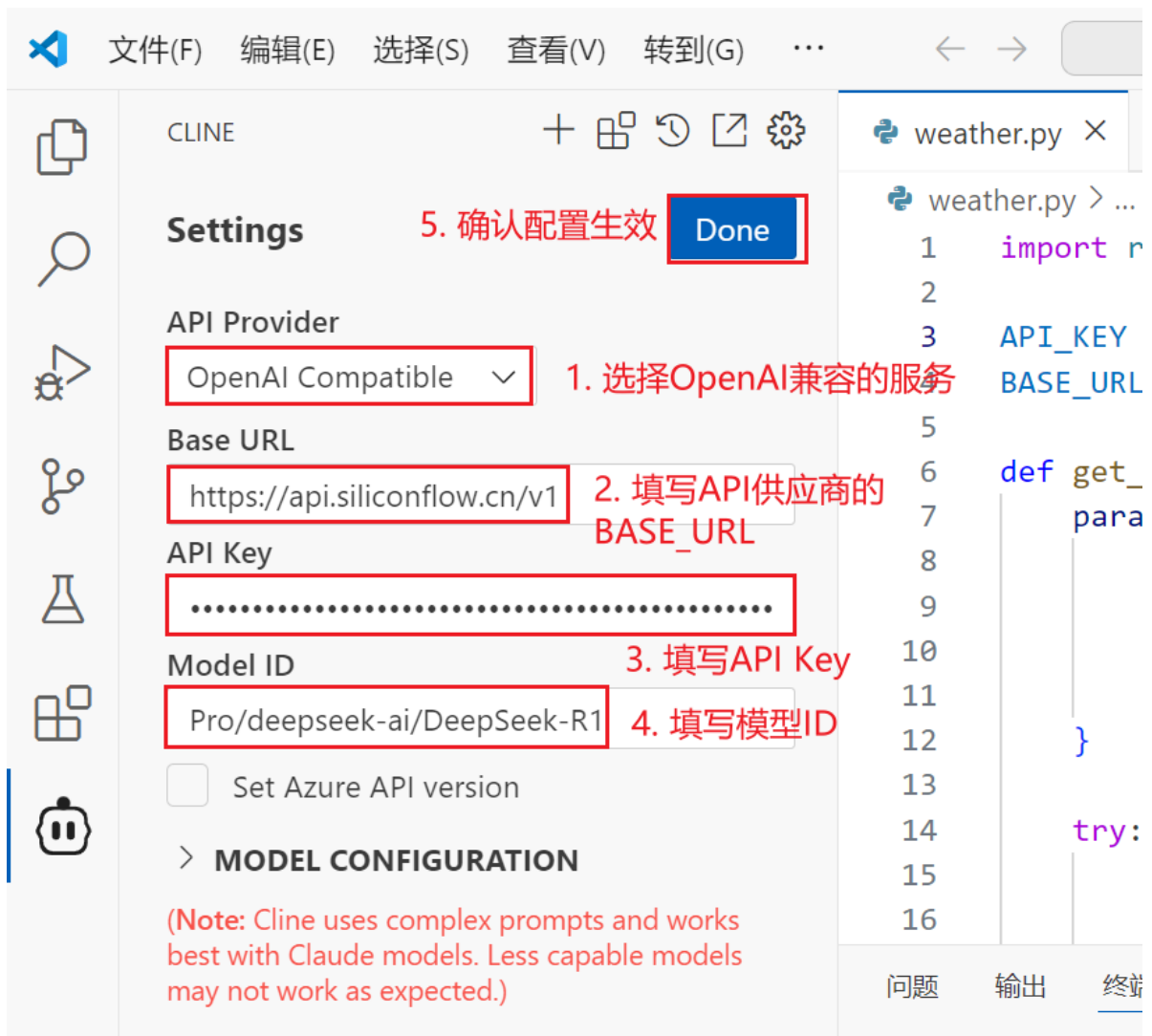
步骤1：插件市场选择Cline，安装即可



步骤2：安装完成后点击左侧图标打开Cline

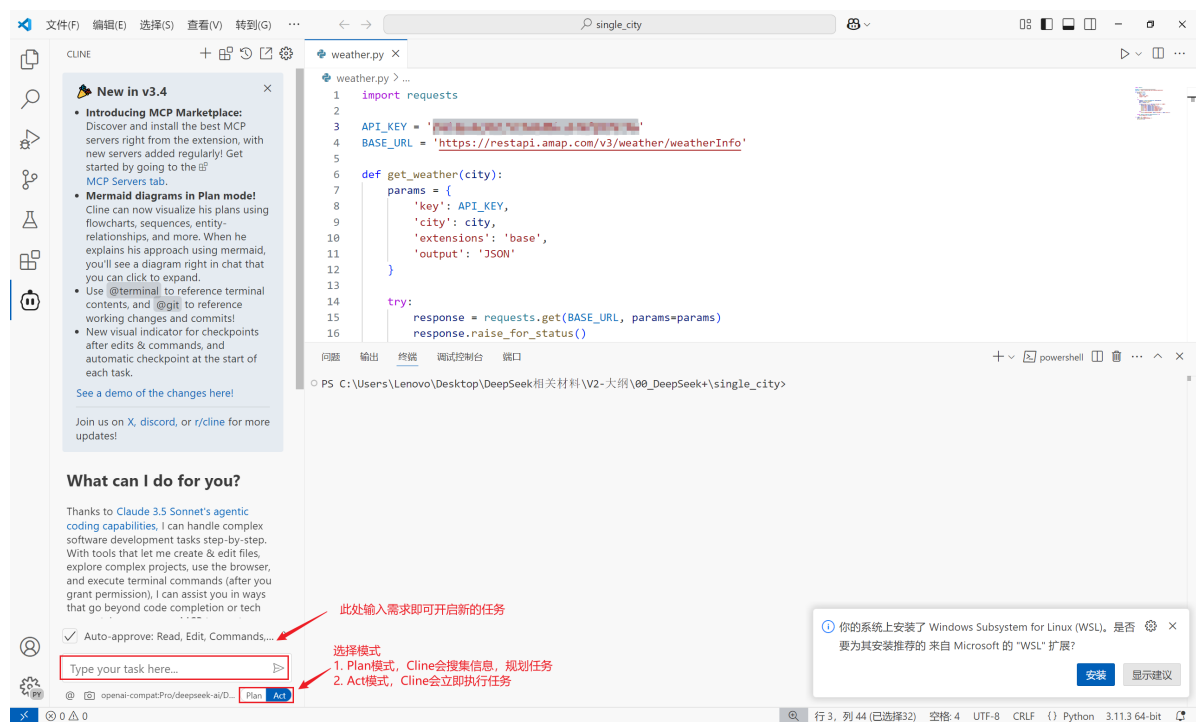


步骤3：点击右上角的齿轮配置API信息



2、Cline使用

2.1 工作模式



2.2 MCP服务

<https://modelcontextprotocol.io/introduction>

MCP服务官网如上，它是Model Context Protocol的简称，是一种由Anthropic推出的开放标准，旨在实现大型语言模型（LLM）与外部数据源和工具之间的无缝集成。MCP通过标准化协议，使AI模型能够安全地访问和操作本地及远程数据，从而提升AI应用的响应质量和工作效率。

2.3 Cline支持的服务

☒ Auto-approve: Read, Edit, Commands... ▾

Auto-approve allows Cline to perform the following actions without asking for permission. Please use with caution and only enable if you understand the risks.

☒ Read files and directories

Allows access to read any file on your computer.

读文件和目录的权限

☒ Edit files

Allows modification of any files on your computer.

创建和编辑文件的权限

☒ Execute safe commands

Allows execution of safe terminal commands. If the model determines a command is potentially destructive, it will still require approval.

执行安全命令的权限

☒ Use the browser

Allows ability to launch and interact with any website in a headless browser.

联网搜索权限

☒ Use MCP servers

Allows use of configured MCP servers which may modify filesystem or interact with APIs.

MCP服务调用权限

Max Requests: 20

最大请求数，是对于单个问题Cline向LLM发送的API请求次数的最大值

Cline will automatically make this many API requests before asking for approval to proceed with the task.

☒ Enable Notifications

Receive system notifications when Cline requires approval to proceed or when a task is completed.

启用提示，当Cline向用户请求执行某些操作或任务完成时允许发送提示

默认全选即可。

最多请求数量可根据实际需求灵活调整。

3、案例一：天气查询项目

3.1 需求说明

在本例中，我们通过Cline编写一个天气查询项目，通过单一的py文件实现

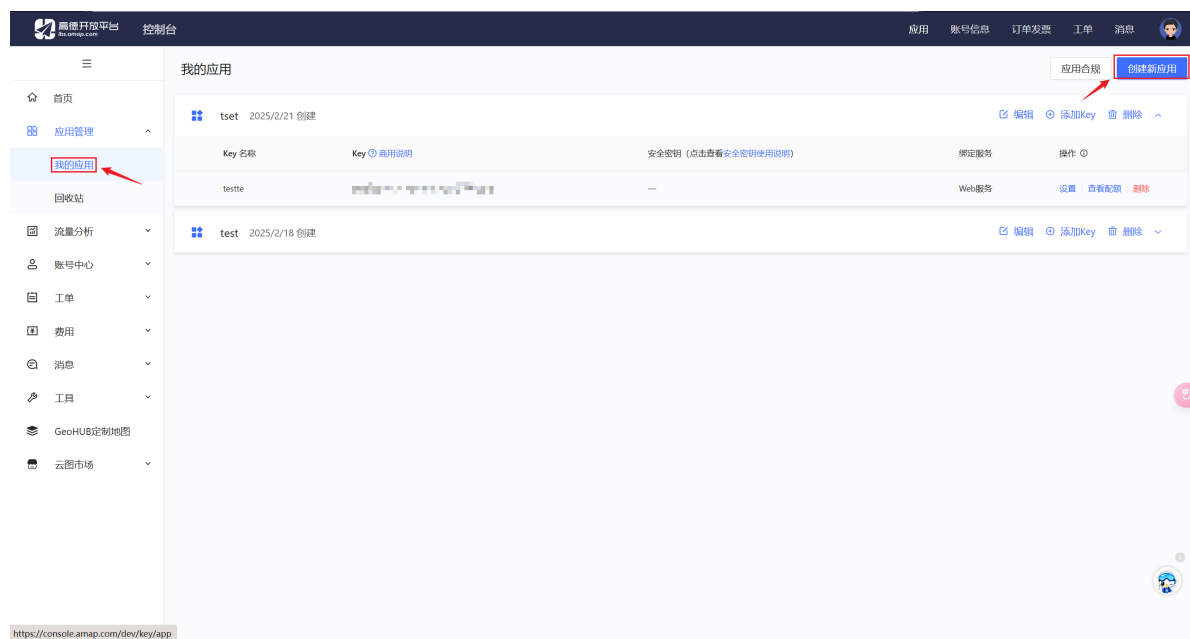
我们需要准备高德地图API Key，操作如下

3.2 申请高德API Key

<https://console.amap.com/>

打开上述网址，按照提示注册（可能需要实名认证）

步骤1：创建应用



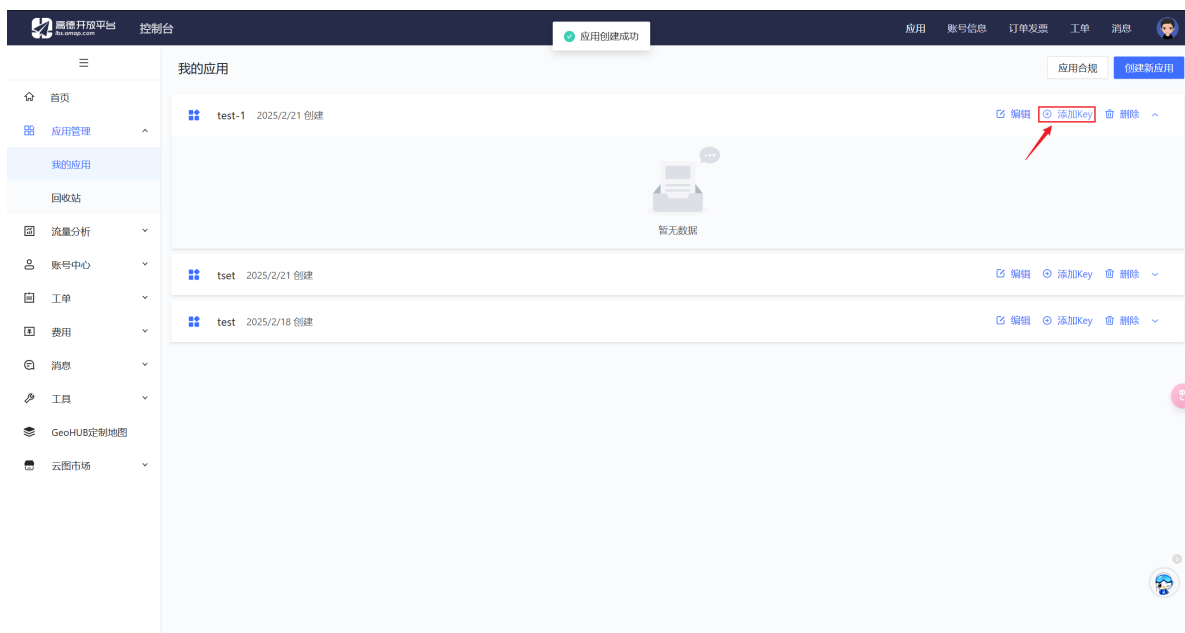
* 应用名称: test-1

* 应用类型: 天气

取消

新建

步骤2：创建API Key



⊕ 为「test-1」添加Key

×

* Key名称:

City-Weather

命名规范

* 服务平台:

Android平台

iOS平台

Web端(JS API)

Web服务

智能硬件

微信小程序

HarmonyOS NEXT 平台

可使用服务:

静态地图API

地理编码API

逆地理编码API

关键字搜索API

周边搜索API

多边形搜索API

ID查询API

输入提示API

路径规划API

坐标转换API

行政区划查询API

IP定位API

天气查询API

矩形区域交通态势API

圆形区域交通态势API

指定线路交通态势API

地理围栏API

猎鹰服务API

GeoHUB服务API

IP白名单:

非必填，留空表示无IP限制

添加IP白名单后，只有白名单中的IP可访问服务

IP应该设定为服务器出口IP，支持设定IP段，如:202.202.2.*，多个IP请每行填写一条

阅读并同意

高德地图开放平台服务协议

和

高德地图开放平台隐私政策

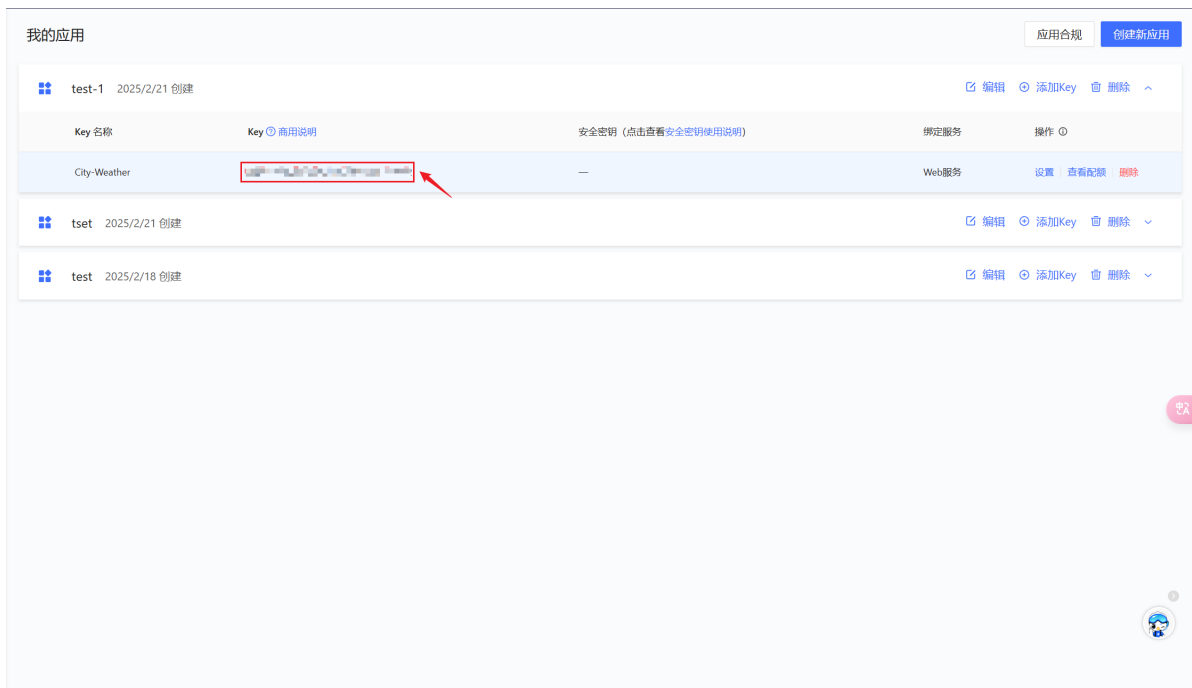
法人或非法人组织使用平台服务应事先购买[商用服务](#)以获取授权。若您未购买商用服务，平台向您提供的KEY和服务配额仅供您用于短期、少量的测试目的；若您超出前述范围使用平台服务（包括但不限于您开始向第三方或公众销售或提供您的应用、将您的应用于参与第三方投标、您的应用在应用程序分发平台上架或已经被公众获取使用、您的应用开始收费或发布广告或以其他方式获益、在组织内部使用的您的应用已上线运营、您的应用长期或大量调用平台服务等情形），构成未授权使用，平台有权采取相应措施。

取消

提交

名称符合规范，提交即可

步骤3：复制此处的API Key即可



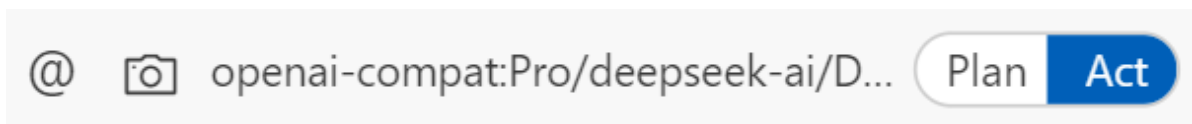
3.3 实操：向Cline下达命令

步骤1：在任意位置创建空目录Weather-1，作为Cline的工作目录

在VSCode中打开该目录。

步骤2：选择Cline工作模式

大多数情况下，直接使用Act模式即可，特别复杂的任务才需要先拆分任务，设计工作流，然后再执行



步骤3：在聊天窗口中写入以下内容发送给Cline

- 1 帮我实现一个weather.py程序，可以通过高德提供的API-key获取城市天气

步骤4：Cline申请执行命令

The screenshot shows the Cline application interface with the following components and annotations:

- Task:** 帮我实现一个weather.py程序，可以通过高德提供的API-key获取城市天气 (See more). *用户的需求*
- Tokens:** ↑ 9.3m ↓ 241.7k. *token用量、上下文窗口范围*
- Context Window:** 4.0m — 128.0k. *API金额的统计信息。不准，可以不看*
- API Cost:** \$0.0000. 45.1 kB.
- Checkpoint:** ----- [Compare] [Restore]
- API Request:** ✓ *Cline向LLM服务器发送API请求*
- Reasoning:** ...认，因为这会修改系统环境。 > *大模型的回复*
- Cline wants to create a new file:** weather.py. *Cline根据大模型的回复调用工具创建文件*
- Checkpoint:** ----- [Compare] [Restore]
- API Request:** ✓ *Cline再次发送请求下面是LLM的回复*
- Reasoning:** ..., 就可以指导用户运行程序。 >
- Cline wants to execute this command:** pip install requests. *Cline希望执行pip命令安装Python的requests包*
- Warning:** ⚠ The model has determined this command requires explicit approval. *模型决策认为该命令属于高风险命令，需要显式申请权限*

此时Cline对话框上方会有弹窗，选择运行命令，pip install将会在命令行执行

 文件(F) 编辑(E) 选择(S) 查看(V) ... < >



CLINE



 Task 

帮我实现一个weather.py程序，可以通过高德提供的API-key获取城市天气 [See more](#)

Tokens: ↑ 9.3m ↓ 241.7k

Context Window: 4.0m 128.0k

API Cost: \$0.0000  45.1 kB

 Cline wants to create a new file:

weather.py 

 Checkpoint -----  Compare  Restore

 API Request 

Reasoning: ..., 就可以指导用户运行程序。 >

 Cline wants to execute this command:

pip install requests

 The model has determined this command requires explicit approval.

☒ Auto-approve: Read, Edit, Commands,... >

Run Command

Reject

Type a message...



@  openai-compat:Pro/deepseek-ai/D...

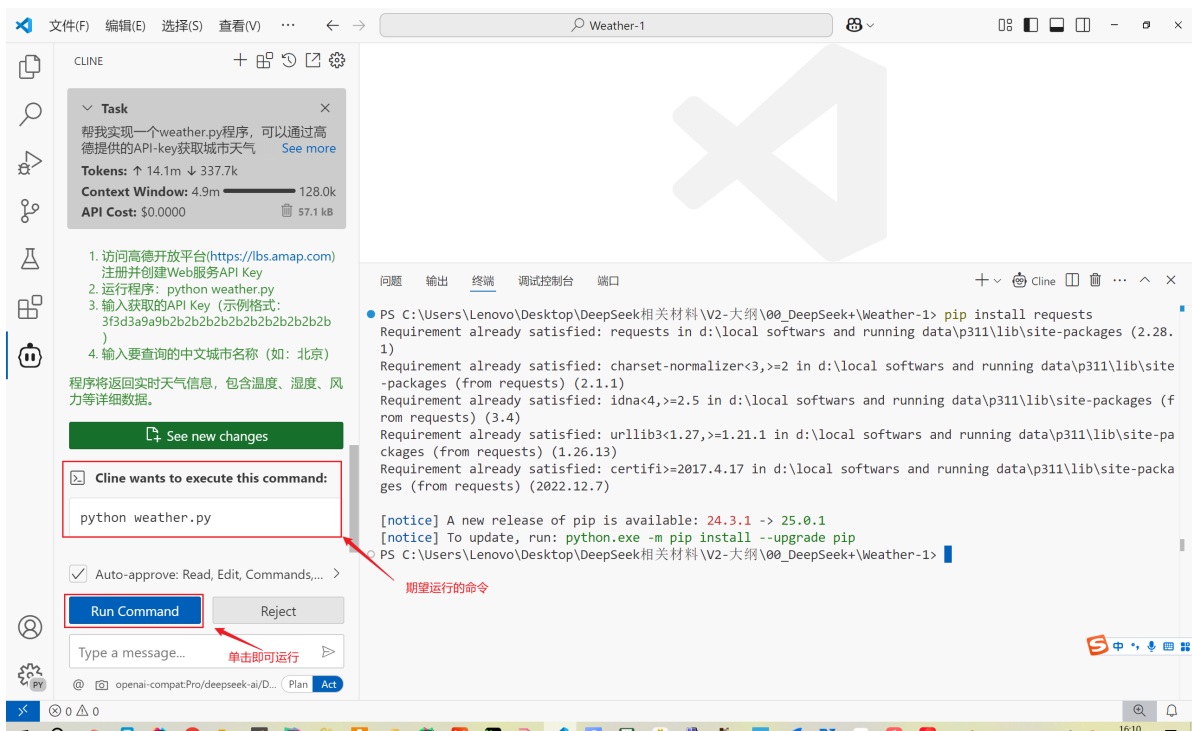
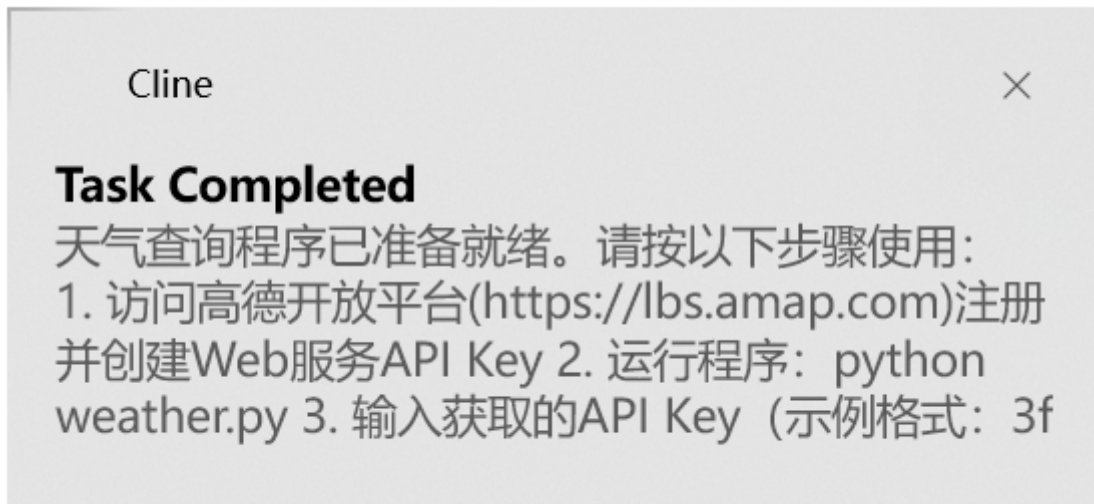
Plan Act

<<

 0  0

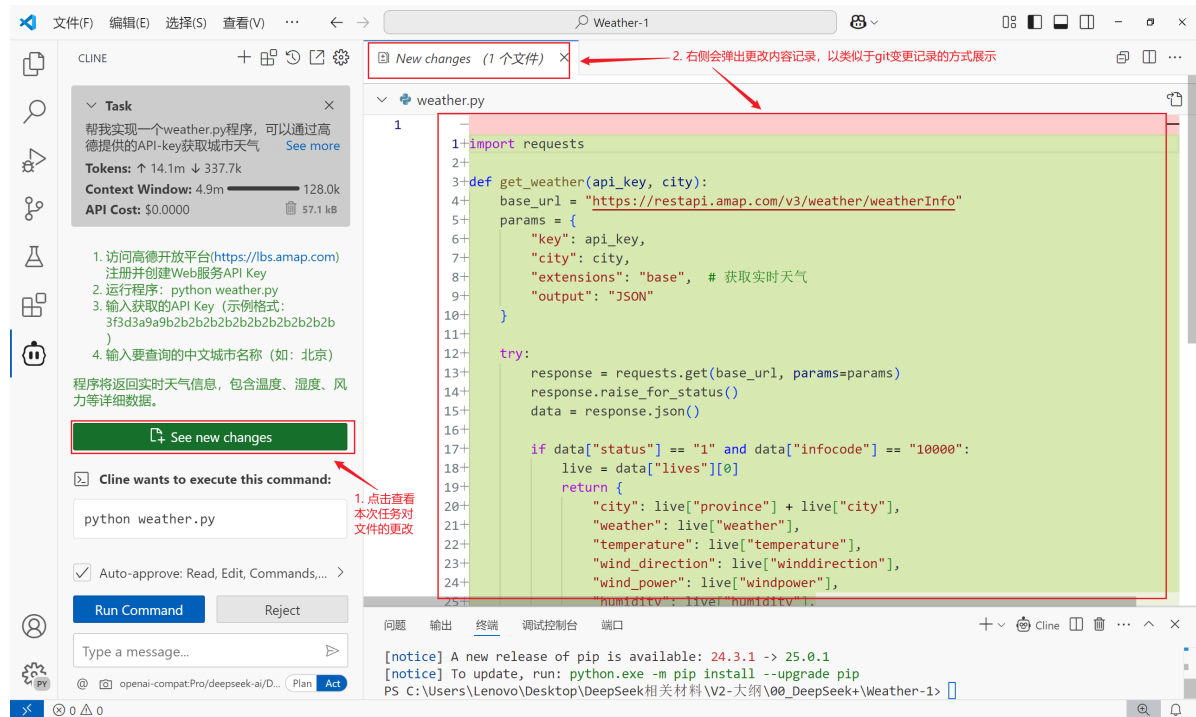
步骤5：Cline提示任务完成，请求运行最后的命令

Windows右下角弹窗提示任务运行完成，并指示程序使用方式

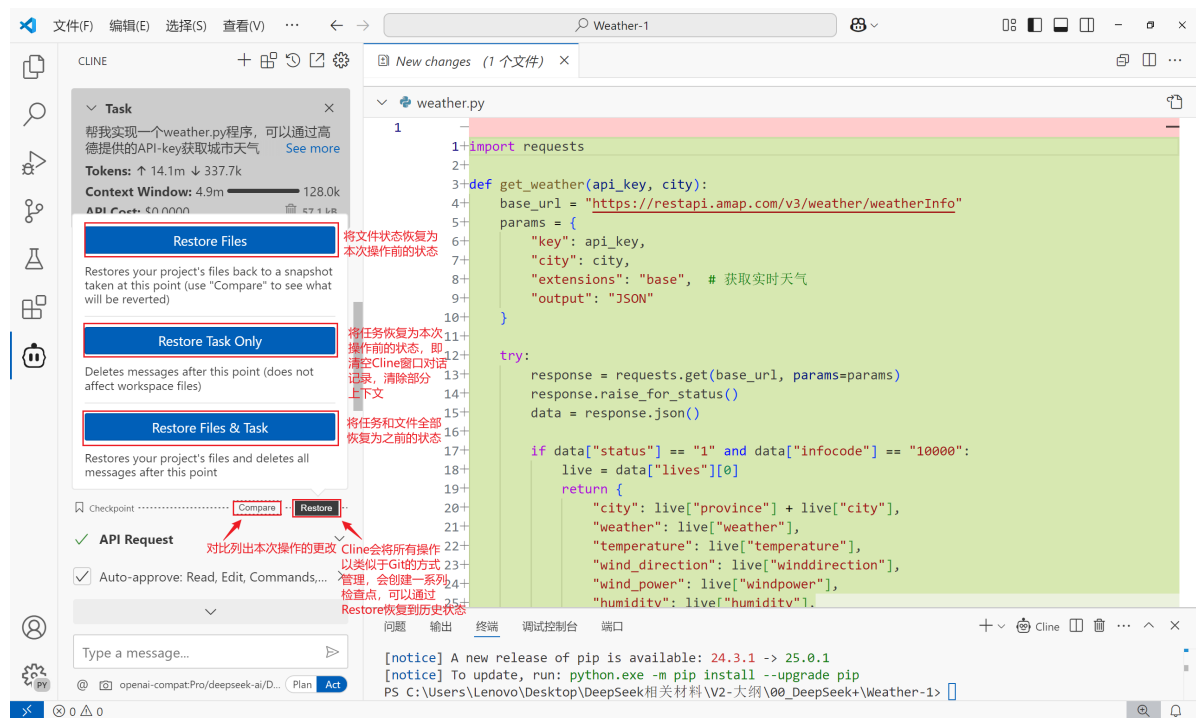


Reject表示拒绝运行，根据实际情况选择。

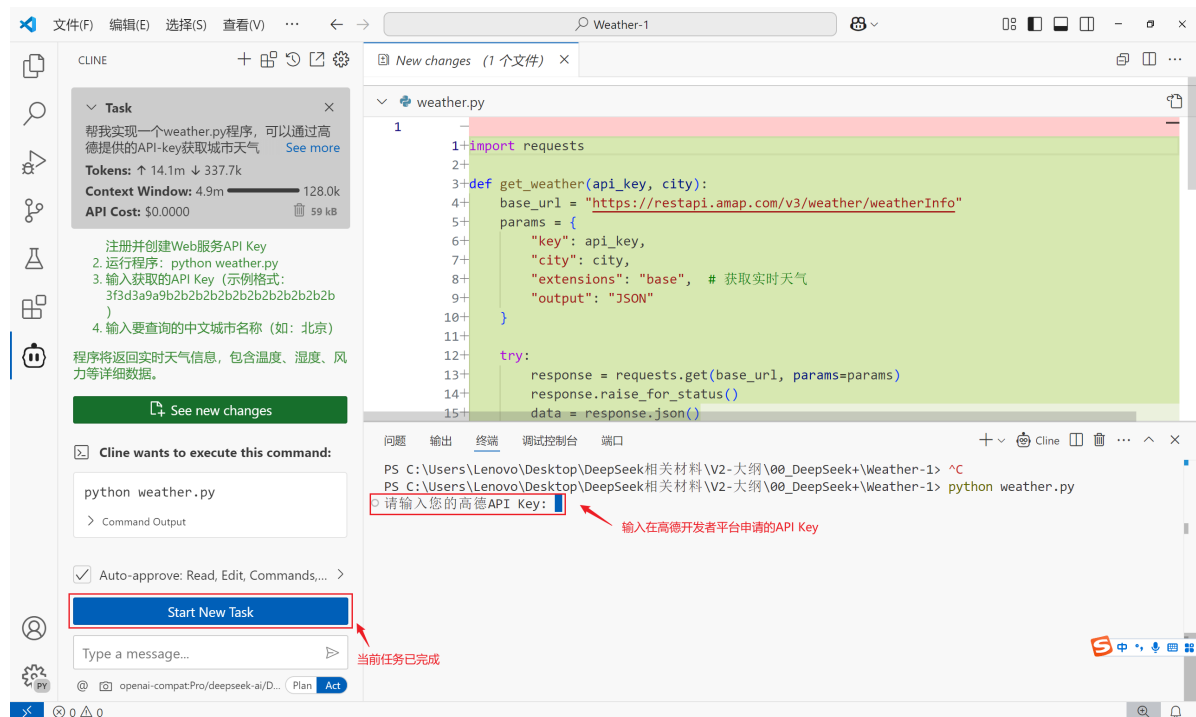
步骤6：查看本次任务执行的文件变更操作



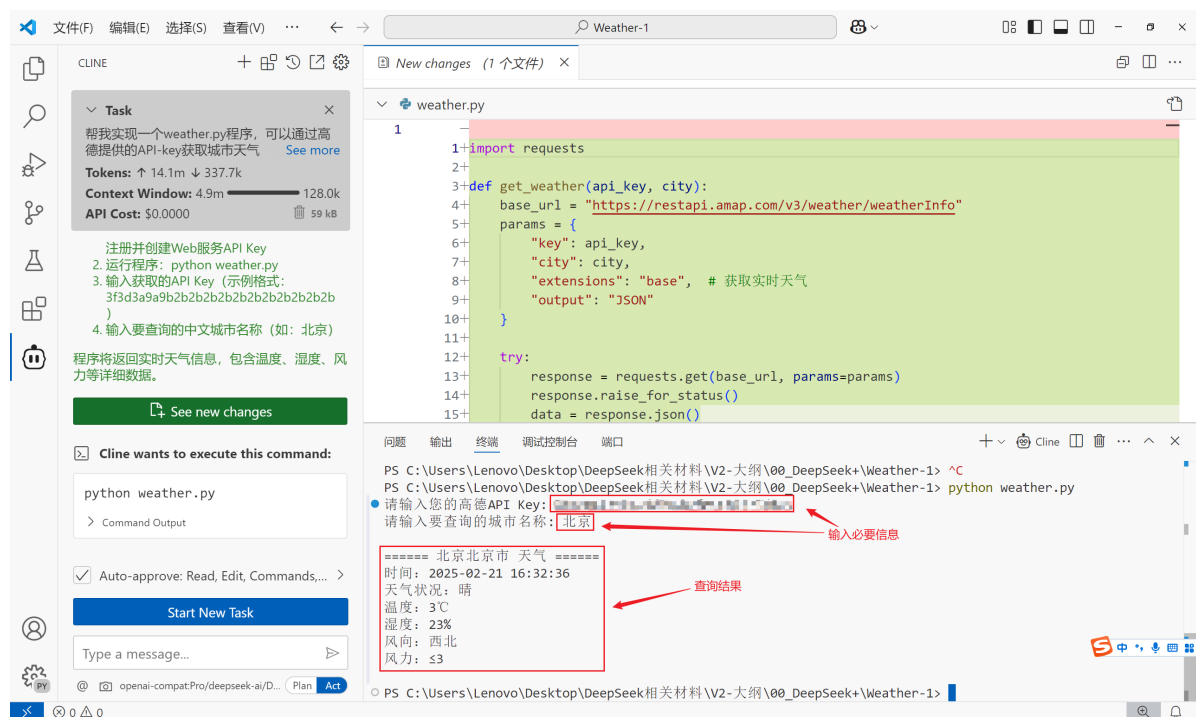
步骤7：查看本次操作变更及撤销变更操作



步骤8：点击运行



步骤9：输入必要信息后的天气查询结果如下



4、案例二：双城天气对比项目

4.1 需求说明

在本例中，我们通过Cline编写一个双城天气对比项目，通过Python Flask框架完成后端，通过HTML、CSS+JavaScript组合完成前端。

目标：在本地7777端口启动Web服务，通过浏览器访问，在前端页面提供两个文本框和一个按钮，输入两个城市名称，点击按钮可以输出两个城市的温差。

提示词如下：

- 1 基于高德提供的天气接口实现一个前后端分离的项目，使用Python Flask框架完成后端开发，使用前端三剑客HTML、CSS和JavaScript完成前端开发。
- 2 目标：在本地7777端口启动Web服务，通过浏览器访问该端口。前端页面提供两个文本框，分别提示用户输入两个待查询城市的名称，提供一个按钮，用户点击按钮时，后端调用天气接口查询双城天气，前端页面分别展示两个城市的天气，要求输出对称。此外，还要大模型基于两个城市的天气查询结果从不同角度对比，在前端展示天气对比概述。布局要尽可能美观。

（本例直接使用上文申请的API Key

4.2 实操

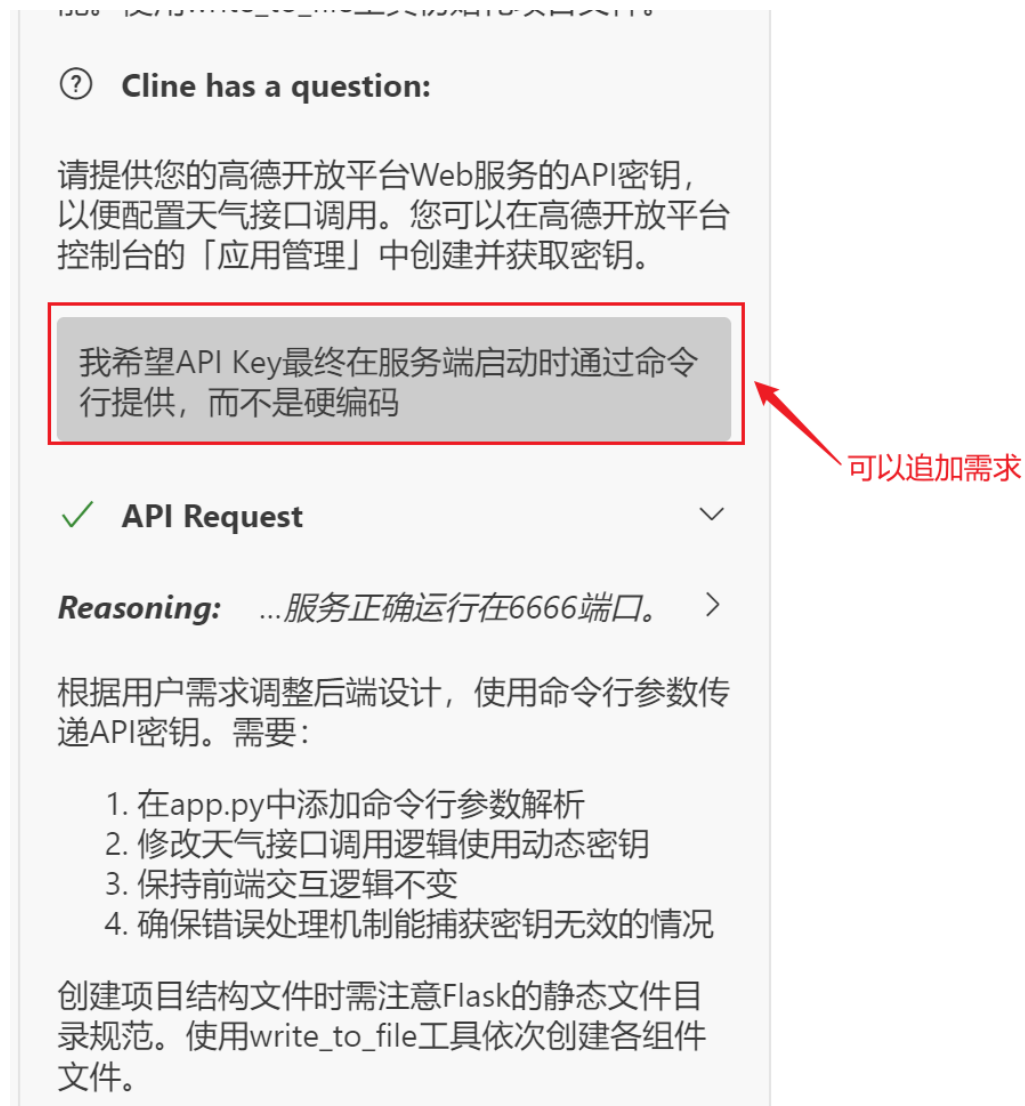
步骤1：在任意目录下创建Weather-2目录，用VSCode打开

步骤2：输入上述提示词

步骤3：生成过程如下

等待时间较长

1. 可以追加需求



② Cline has a question:

请提供您的高德开放平台Web服务的API密钥，以便配置天气接口调用。您可以在高德开放平台控制台的「应用管理」中创建并获取密钥。

我希望API Key最终在服务端启动时通过命令行提供，而不是硬编码

✓ API Request

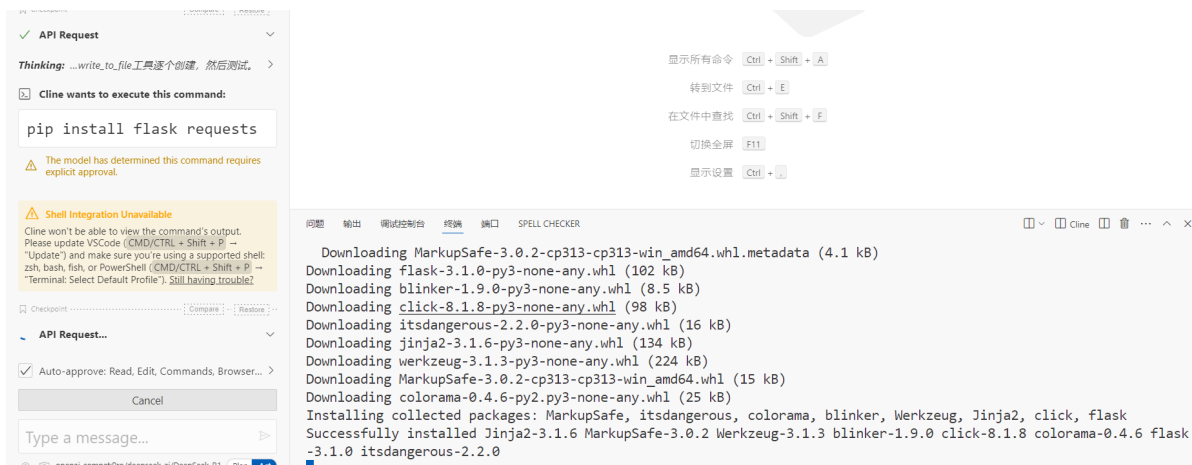
Reasoning: ...服务正确运行在6666端口。 >

根据用户需求调整后端设计，使用命令行参数传递API密钥。需要：

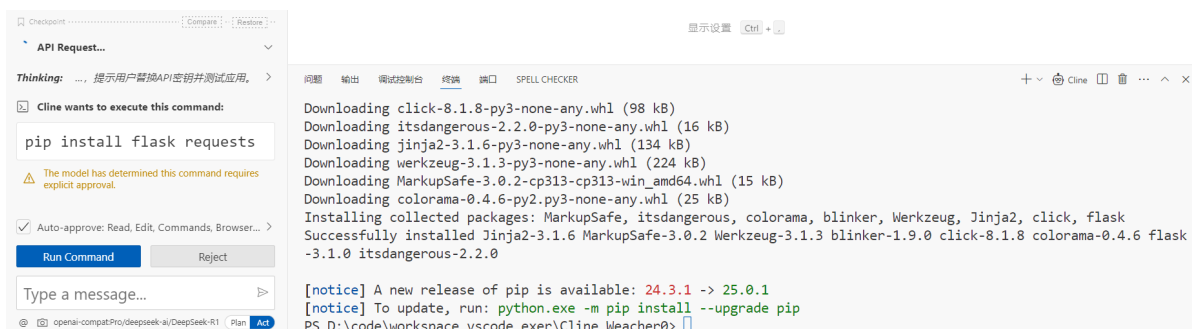
1. 在app.py中添加命令行参数解析
2. 修改天气接口调用逻辑使用动态密钥
3. 保持前端交互逻辑不变
4. 确保错误处理机制能捕获密钥无效的情况

创建项目结构文件时需注意Flask的静态文件目录规范。使用write_to_file工具依次创建各组件文件。

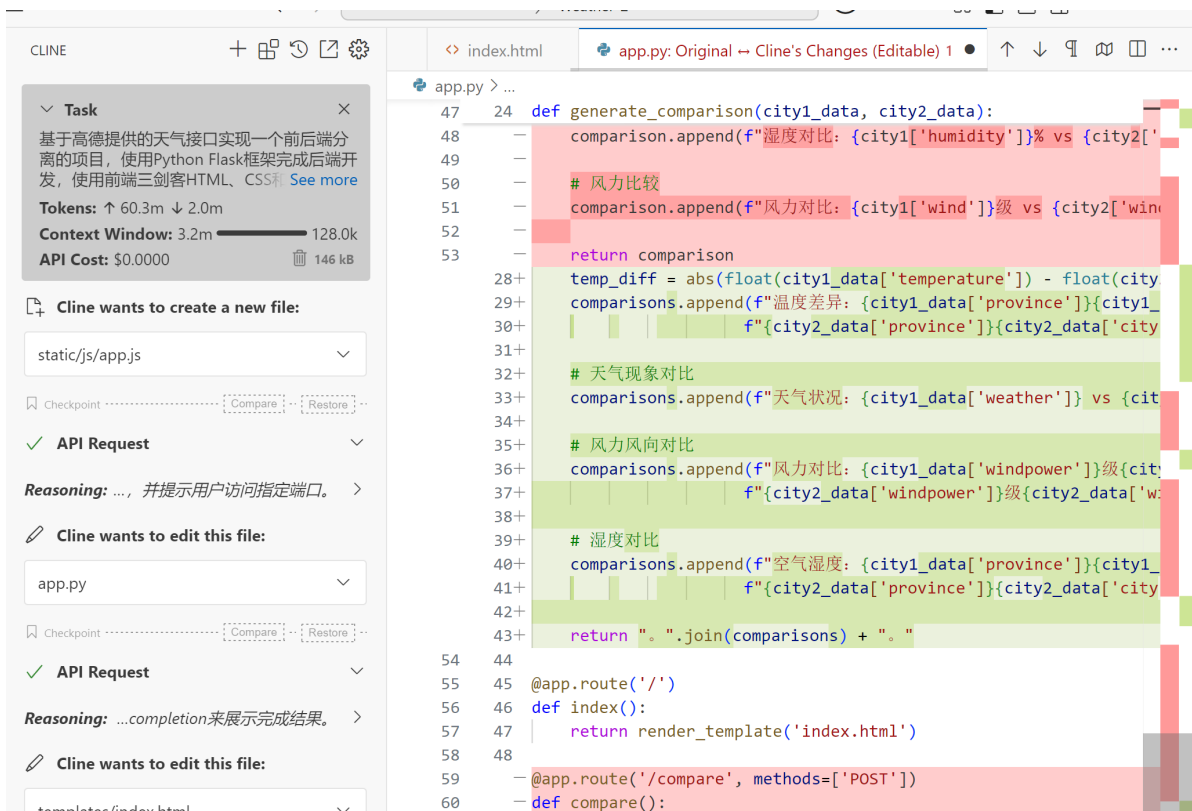
可以追加需求



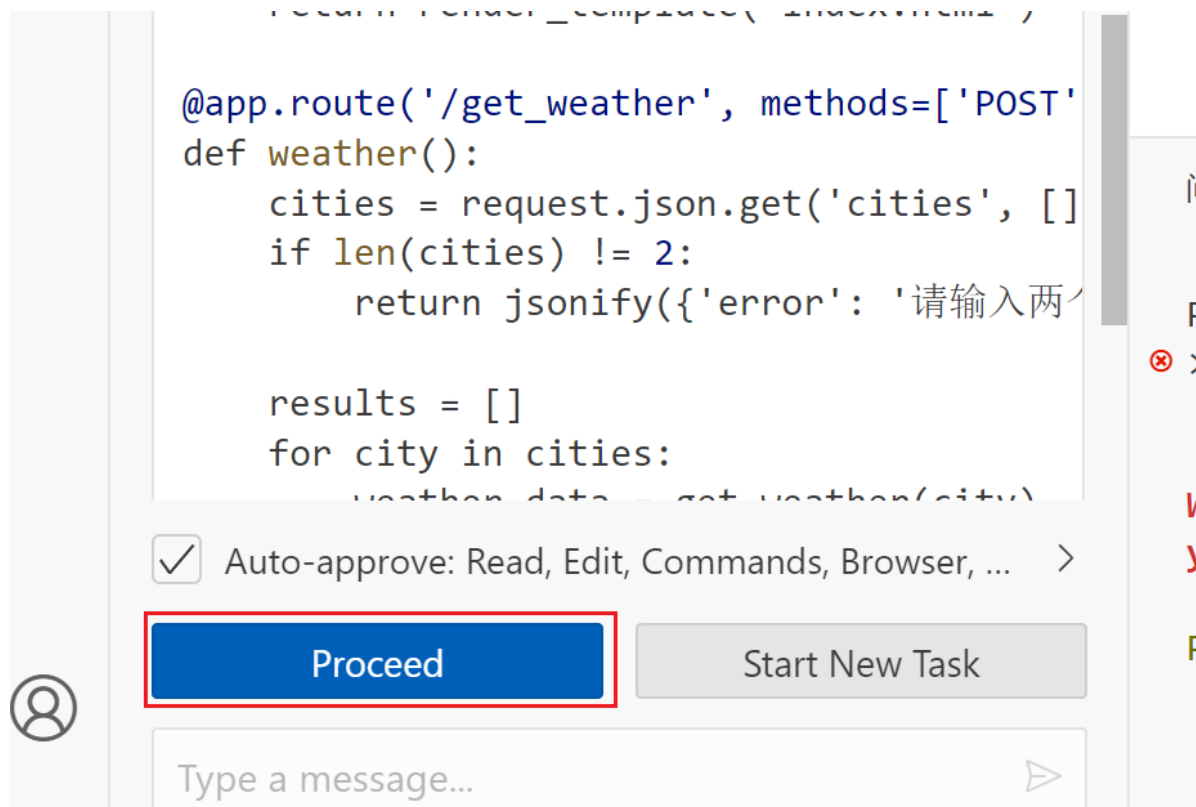
期间会下载flask框架，如上图。会下载flask requests，如下图：



2. 在输出最终结果之前，Cline可能会校验代码，如果发现潜在的bug可能会做出更改，如下。



3. 到达设定的交互上限工作会暂停，询问用户继续还是开启新任务



4. 本次任务日志如下

Task



基于高德提供的天气接口实现一个前后端分离的项目，使用Python Flask框架完成后端开发，使用前端三剑客HTML、CSS和JavaScript完成前端开发。
目标：在本地6666端口启动Web服务，通过浏览器访问该端口。前端页面提供两个文本框，分别 [See more](#)

Tokens: ↑ 192.9m ↓ 6.9m

Context Window: 10.6m 128.0k

API Cost: \$0.0000 399 kB

以下是代码清单，请按照清单中的要求完成开发：

- 1. 后端服务 (app.py)
- 2. 前端页面 (templates/index.html)
- 3. 样式文件 (static/css/style.css)
- 4. 前端逻辑 (static/js/app.js)

需要获取高德天气API的授权密钥，使用ask_followup_question工具获取该关键信息。同时需要处理以下技术点：

- 前后端数据交互格式 (JSON)
- 高德API调用参数处理
- 双城天气数据对比逻辑
- 响应式页面布局设计
- 错误处理机制

优先创建基础项目结构，然后逐步补充各模块功能。使用write_to_file工具初始化项目文件。

Cline has a question:

请提供您的高德开放平台Web服务的API密钥，以便配置天气接口调用。您可以在高德开放平台控制台的「应用管理」中创建并获取密钥。

我希望API Key最终在服务端启动时通过命令行提供，而不是硬编码

API Request



Reasoning

好的，用户希望API Key通过命令行参数传入而不是硬编码在代码中。我需要调整后端代码设计，使用Python的argparse模块来处理命令行参数。首先，在app.py中需要添加命令行参数解析逻辑，确保启动服务时必须传入有效的API密钥。同时，在前端部分保持原有设计，通过POST请求将城市名称发送到后端。需要修改Flask应用的初始化部分，将API密钥作为配置参数。在调用高德接口时，从应用配置中获取密钥。这样既保证安全性，又符合用户的需求。接下来，需要创建四个核心文件：后端主程序、前端HTML模板、CSS样式表和4. 前端逻辑 (static/js/app.js)

需要获取高德天气API的授权密钥，使用ask_followup_question工具获取该关键信息。同时需要处理以下技术点：

- 前后端数据交互格式 (JS

Task



基于高德提供的天气接口实现一个前后端分离的项目，使用Python Flask框架完成后端开发，使用前端三剑客HTML、CSS和JavaScript完成前端开发。
目标：在本地6666端口启动Web服务，通过浏览器访问该端口。前端页面提供两个文本框，分别 [See more](#)

Tokens: ↑ 192.9m ↓ 6.9m

Context Window: 10.6m 128.0k

API Cost: \$0.0000 399 kB

Checkpoint

Compare

Restore

API Request



Reasoning

好的，我现在需要帮用户实现一个基于高德天气接口的前后端分离项目。用户要求使用Python Flask作为后端，HTML、CSS和JavaScript作为前端。首先，我需要明确项目的结构，然后逐步实现各个模块的功能。我会先创建基础项目结构，然后逐步补充各模块功能。使用write_to_file工具初始化项目文件。

好的，我现在需要帮用户实现一个基于高德天气接口的前后端分离项目。用户要求使用Python Flask作为后端，前端用HTML、CSS和JavaScript，还要在本地6666端口启动服务。前端要有两个输入框，一个按钮，点击后显示两个城市的天气对比和大模型的对比分析。布局要美观。首先，我得考虑项目结构。因为当前工作目录下没有文件，所以需要从头开始搭建。通常Flask项目会有后端和前端分开的目录，比如backend和static/templates。不过用户可能希望前后端分离更明确，所以可能需要分别创建前端和后端的文件。后端部分，我需要用Flask创建API接口，调用高德的天气接口。首先得注册高德开放平台获取API密钥。然后编写一个路由，接收两个城市名称，调用高德的天气接口获取数据，处理后再返回给前端。同时，还需要处理可能的错误，比如城市不存在或者API调用失败的情况。前端部分，需要两个输入框和一个按钮，点击按钮时通过JavaScript调用后端的API。获取数据后，用对称的方式展示两个城市的天气信息，比如并排的两个卡片。然后还需要调用大模型的API来生成对比分析，这部分可能需要额外的后端处理，或者在前端直接调用大模型的API。不过用户要求后端调用天气接口，所以对比分析可能也需要后端来处理，使用大模型的API，比如OpenAI的接口，或者本地的大模型服务。不过用户没有提到大模型的具体实现方式，可能需要进一步澄清。但根据任务描述，可能是在后端获取天气数据后，由后端生成对比分析，然后一起返回给前端。所以后端需要集成天气API和大模型的处理。接下来，我需要考虑文件结构。创建backend目录存放Flask应用，static目录放CSS和JavaScript，templates目录放HTML文件。不过Flask默认的静态文件目录是static，模板目录是templates，所以需要按照这个结构来组织。然后，编写后端的app.py，配置路由，处理请求。前端需要index.html，样式表style.css，以及脚本app.js。还需要考虑跨域问题，不过因为前后端都在同一个域下，可能不需要处理CORS。高德的天气接口需要注册获取key，这部分需要用户提供，或者让用户自己注册。但任务中可能需要引导用户获取API密钥，或者假设已经有可用的key。但用户可能没有提供，所以可能需要使用ask_followup_question工具来询问用户的高德API密钥。另外，大模型的对比分析部分，用户没有明确说明使用哪个模型，如何调用。可能需要假设使用OpenAI的API，或者让用户提供自己的API密钥。这部分也需要进一步澄清，但根据任务描述，可能需要在后端处理，所以需要添加相应的代码。现在，我需要逐步创建文件。首先创建后端文件，比如app.py，然后前端的三剑客文件。同时，需要确保Flask正确配置静态文件和模板目录。在编写代码时，后端需要处理两个城市名称，分别调用高德接口，获取天气数据，然后生成对比分析，可能调用大模型API，最后将数据返回给前端。前端则通过JavaScript获取数据，动态更新页面显示。布局方面，使用CSS Flexbox或者Grid来实现对称布局，确保两个城市的信息并排显示，对比分析在下方。样式要美观，可能需要设置卡片样式、背景颜色、字体等。测试方面，启动Flask应用后，打开浏览器访问localhost:6666，输入城市名称，点击按钮查看结果。需要确保错误处理得当，比如城市输入错误时提示用户。可能遇到的问题：高德API的调用方式、跨域问题、大模型API的集成、前端动态内容渲染。需要逐步解决。使用工具创建文件、测试，并根据结果调整代码。

```
try:
    cities = request.json.get('cities', [])
    if len(cities) != 2:
        return jsonify({"error": "需要两个城市进行对比"}), 400

    results = []
    for city in cities:
        params = {
            "key": app.config['API_KEY'],
            "city": city,
            "extensions": "base",
            "output": "JSON"
        }
        response = requests.get(API_BASE, params=params)
        data = response.json()

        if data["status"] != "1":
            return jsonify({"error": f"获取{city}天气失败: {data.get('info', '')}"})

        live = data["lives"][0]
        results.append({
            "city": city,
            "weather": live["weather"],
            "temperature": live["temperature"],
            "windpower": live["windpower"],
            "humidity": live["humidity"]
        })

    # 生成对比分析
    analysis = generate_comparison(results[0], results[1])
    return jsonify({"cities": results, "analysis": analysis})

except Exception as e:
    return jsonify({"error": str(e)}), 500

def generate_comparison(city_a, city_b):
```

```

comparisons = []

# 温度对比
temp_diff = abs(int(city_a["temperature"]) - int(city_b["temperature"]))
comparisons.append(f"温度差异: {city_a['city']} ({city_a['temperature']}°C)

# 天气状况
if city_a["weather"] == city_b["weather"]:
    comparisons.append(f"天气状况相同: 均为{city_a['weather']}")
else:
    comparisons.append(f"天气差异: {city_a['city']}是{city_a['weather']}, 而{

# 风力对比
wind_comparison = "风力相当" if city_a["windpower"] == city_b["windpower"] e
    f"{city_a['city']}风力较大 ({city_a['windpower']})" if city_a["windpower
    f"{city_b['city']}风力较大 ({city_b['windpower']})"
comparisons.append(wind_comparison)

# 湿度对比
humidity_diff = abs(int(city_a["humidity"][:-1]) - int(city_b["humidity"][:
comparisons.append(f"湿度差异: {humidity_diff}%")

return comparisons

if __name__ == '__main__':
    app.run(port=6666, debug=True)

```

</final_file_content>

IMPORTANT: For any future changes to this file, use the final_file_content show

Tool [write_to_file] was not executed because a tool has already been used in t

```

<environment_details>
# VSCode Visible Files
    </div>
    <div class="city-card" id="city2-result">
        <h2>城市二</h2>
        <div class="weather-info"></div>
    </div>
</div>
</div>

    <script src="{ { url_for('static', filename='js/app.js') } }"></script>
</body>
</html>

</final_file_content>

```

IMPORTANT: For any future changes to this file, use the final_file_content show

失败, 返回相应的错误信息给前端。最后, 将所有文件组织到正确的目录结构中, 比如static/css和static/js目录, 并确保Flask正确配置静态文件路径。运行Flask应用后, 用户可以通过访问本地6666端口来使用系统。

❓ Cline has a question:

请提供您的高德地图API密钥 (Web服务API Key), 这是调用天气接口的必要凭证。您可以在高德开放平台 (<https://console.amap.com>) 的应用管理中找到。

我希望启动服务端时通过命令行提供API Key, 而非硬编码, 上文已经提过了, 你的服务端代码也是按照这

✓ API Request

app.py

Checkpoint

Compare

Restore

✓ API Request

Reasoning

好的，我现在需要帮助用户完成一个基于高德天气接口的双城天气对比项目。用户已经提供了一些代码文件，但似乎在前端HTML文件的创建过程中被中断了。首先，我需要确认用户的需求是否已经全部完成，或者还有哪些部分需要补充。首先，用户的后端代码app.py已经创建，包含了获取天气数据和生成对比分析的逻辑。前端JavaScript文件app.js也已完成，处理用户输入和数据的展示。但HTML文件templates/index.html的创建被中断，只写到了引入CSS的部分，没有完成整个结构。此外，CSS文件static/css/style.css可能还未创建，这会影响页面的美观布局。接下来，我需要补全HTML文件的内容，确保包含两个输入框、按钮以及展示天气信息和对比分析的区域。同时，需要创建CSS文件来定义页面样式，使布局美观对称。还要检查后端API密钥的设置是否正确，确保用户知道如何配置环境变量。另外，用户之前尝试执行的命令中包含了设置环境变量和启动Flask应用，但需要确认用户是否已经拥有高德API密钥，并在执行命令时正确替换。如果用户尚未获取API密钥，可能需要引导他们获取。最后，确保所有文件结构正确，比如Flask的templates和static目录是否正确替换，或者是否需要再次询问用户。查看最新的`app.py`内容，发现密钥仍然是`YOUR\_AMAP\_API\_KEY`，说明替换未成功，因此需要再次使用`replace\_in\_file`工具来更新密钥。然后，需要创建前端页面`templates/index.html`和JavaScript文件`static/js/app.js`。这些文件负责用户界面和与后端的交互。确保HTML结构符合CSS样式，并且JavaScript能够正确调用后端的API，处理用户输入和显示结果。在编写前端代码时，需要注意以下几点：1. HTML结构要包含两个输入框和一个按钮，以及显示天气信息和对比的区域。2. JavaScript需要处理按钮点击事件，获取输入的城市名称，通过Fetch API调用后端的`/get\_weather`接口，处理返回的数据并更新页面。3. 添加加载状态提示，提升用户体验。4. 错误处理，比如输入为空或API返回错误时的提示。另外，需要确保所有文件的路径正确，比如Flask的模板目录`templates`和静态文件目录`static`是否符合默认配置，避免出现404错误。最后，替换API密钥后，需要指导用户如何运行应用，使用`execute\_command`启动Flask服务，并通过浏览器访问验证功能是否正常。可能遇到的问题：- 用户未正确提供API密钥，导致天气数据无法获取。- 前端JavaScript代码中的API端点是否正确指向本地服务器的6666端口。- 跨域问题，但由于前后端同源，应该不存在这个问题。- 样式文件是否正确链接，确保页面布局美观。需要逐步解决这些问题，确保每个部分都能正常工作，最终完成项目目标。

Cline wants to edit this file:

app.py

Checkpoint

Compare

Restore

✓ API Request

Reasoning

好的，我现在需要帮助用户完成一个基于高德天气API的双城天气对比项目。用户已经提供了API密钥，并且我已经创建了部分文件，但还需要完成前端页面和JavaScript逻辑。让我仔细分析当前的情况。首先，用户的目标是创建一个前后端分离的应用，前端使用HTML、CSS和JavaScript，后端使用Flask。根据环境详情，用户已经创建了app.py、style.css，但templates/index.html和static/js/app.js可能还不完整或需要修改。接下来，我需要确保前端页面（index.html）结构正确，包含两个输入框、一个按钮，以及显示天气信息和对比的区域。同时，JavaScript部分需要处理用户输入，调用后端API，并更新页面内容。查看之前的操作，用户尝试写入templates/index.html但被中断。因此，我需要继续完成这个文件的创建。同时，需要创建或更新static/js/app.js来处理前端逻辑。另外，需要确保所有文件路径正确，比如Flask的模板目录是否正确，静态文件路径是否配置正确。根据Flask的默认设置，templates目录用于存放HTML模板，static目录用于CSS、JS和图片等资源，这似乎已经符合当前的文件结构。在JavaScript部分，需要实现以下功能：1. 获取用户输入的两个城市名称。2. 发送POST请求到后端的/get_weather接口。3. 处理返回的天气数据，更新页面显示。4. 显示对比结果。5. 处理加载状态和错误提示。同时，需要考虑用户体验，比如输入验证、加载状态提示、错误处理等。CSS已经由用户提供，但可能需要微调以确保布局美观。现在需要逐步完成这些步骤，使用write_to_file工具创建或更新必要的文件，确保所有部分协同工作。最后，通过execute_command运行Flask应用，测试整个系统是否正常。

Cline wants to edit this file:

templates/index.html

Checkpoint

Compare

Restore

⚠️ **Maximum Requests Reached**

Cline has auto-approved 20 API requests. Would you like to reset the count and proceed with the task?

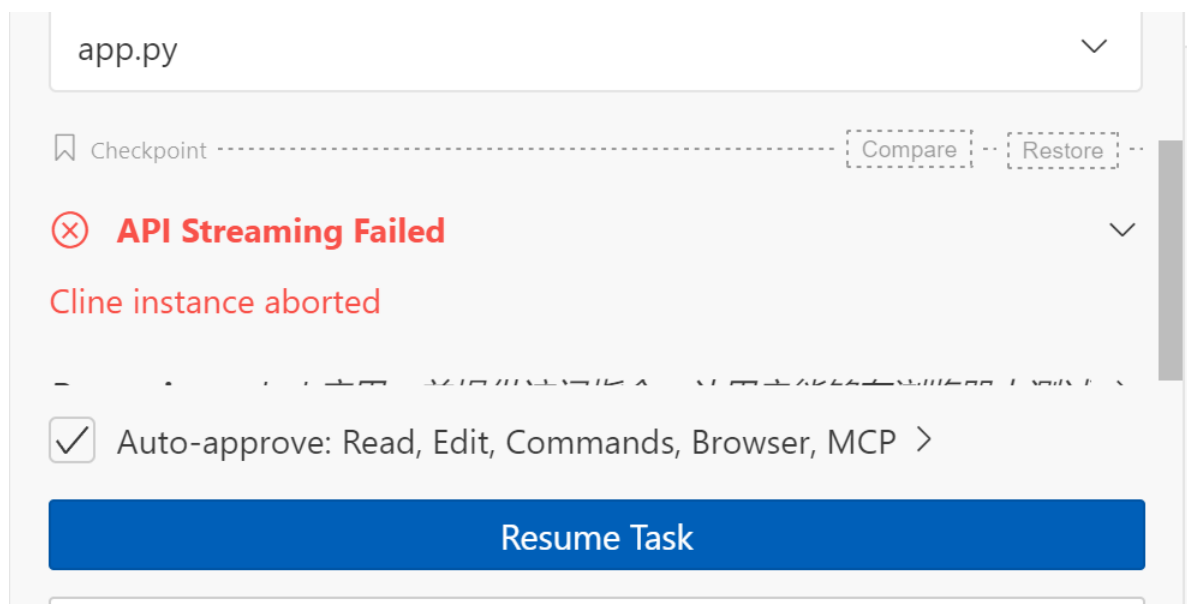
5. 继续当前任务

点击Proceed继续当前任务

6. 阅读日志，干预 workflow

阅读日志发现，Task在不断重复需求，重复的原因都是文件修改被中断，有可能是网络环境差或传输数据太多导致传输中断，此时可以考虑追加提问，基于已完成的部分更改需求或者更改模型，选择速度更快的DeepSeek-V3

点击取消打断 workflow，如下

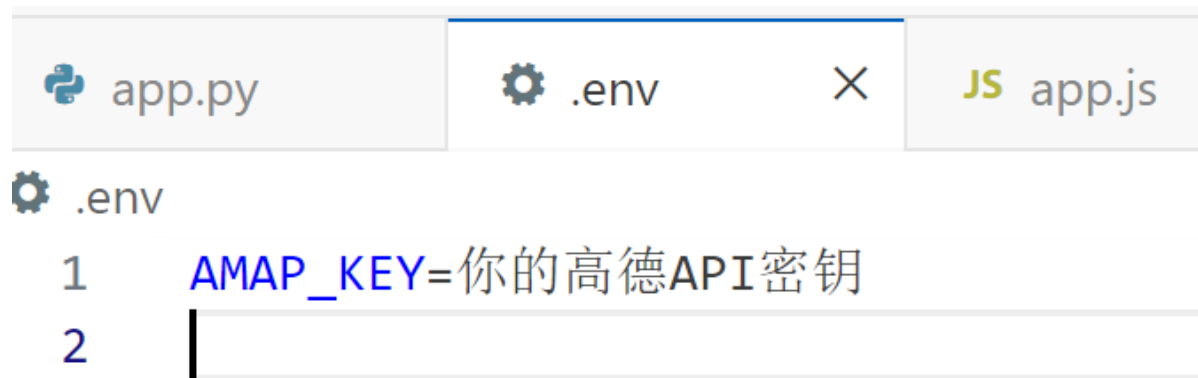


重新组织提示词

- 1 目前你已经帮我完成了代码框架，现在只需要运行代码测试，然后找问题即可，不要再一次次重复生成了

通过python app.py 启动服务端，在浏览器测试运行状态。根据测试结果更新提示词

经过多轮交互，Cline创建了.env文件，只要将下图中的API Key替换为真正的Key即可



7. 剩余工作流日志如下

Task

基于高德提供的天气接口实现一个前后端分离的项目，使用Python Flask框架完成后端开发，使用前端三剑客HTML、CSS和JavaScript完成前端开发。
目标：在本地6666端口启动Web服务，通过浏览器访问该端口。前端页面提供两个文本框，分别 [See more](#)
Tokens: ↑ 371.7m ↓ 11.7m
Context Window: 5.1m 128.0k
API Cost: \$0.0000 817 kB

✓ API Request

Reasoning: ...口运行。最后，使用attempt_completion工具来通知用户服务已启动，并提供访问的URL。 >

✎ Cline wants to edit this file:

app.py

✗ Diff Edit Failed

This usually happens when the model uses search patterns that don't match anything in the file. Retrying...

✓ API Request

Reasoning: ...正确，但根据提供的文件内容，已经是7777，所以不需要修改代码，只需调整启动命令。 >

📄 Cline wants to execute this command:

```
set FLASK_APP=app.py && flask run
```

> Command Output

🔖 Checkpoint

Compare

Restore

✓ API Request

Reasoning: ...有依赖项已安装，如Flask和python-dotenv。如果用户尚未安装这些包，应提供安装命令。 >

用户尝试运行Flask应用时遇到PowerShell命令语法错误。错误显示[&&]在PowerShell中不是有效的语句分隔符。需要调整命令格式以适应PowerShell环境，同时补充缺失的端口参数和环境变量配置。

步骤分析：

- 修正PowerShell命令语法：使用分号[;]替代CMD的[&&]连接符
- 添加明确的端口参数[--port=6666]以满足项目需求
- 确保已安装必要依赖（Flask、python-dotenv）

API Cost: \$0.0000

817 kB

```
app.py
static/js/app.js
templates/index.html

# Current Time
2025/2/21 下午6:16:06 (Asia/Shanghai, UTC+8:00)

# Current Mode
ACT MODE
</environment_details>
```

Reasoning: ...测试。如果一切配置正确，应用应该能够正常处理请求并返回预期的天气数据对比结果。 >

 **Cline wants to execute this command:**

```
python -c "import os; print('AMAP_KEY exists:', 'AMAP_KEY' in os.environ)"
```

> Command Output

 Checkpoint

 Compare

 Restore

✓ **API Request**

Reasoning: ...测试是否解决问题。如果问题依旧，可能需要进一步检查密钥的有效性或网络连接问题。 >

根据执行结果，AMAP_KEY环境变量确实未设置。需要创建.env文件并添加高德地图的API密钥。同时需要确保：

1. 用户已申请高德开放平台的Web服务API Key
2. 密钥已正确配置到.env文件中
3. Flask应用能正确加载环境变量

现在需要指导用户完成这些步骤。首先创建.env文件，然后验证密钥有效性。

 **Cline wants to create a new file:**

.env

app.py

⊗ **Diff Edit Failed**

This usually happens when the model uses search patterns that don't match anything in the file. Retrying...

✓ **API Request**

Reasoning: ...确保前端发送的请求格式与后端匹配。可能还需要指导用户正确设置.env文件或环境变量。 >

根据当前代码分析，问题出在前后端数据格式不匹配。前端发送的是表单数据而后端期望JSON格式。需要统一数据格式并修复API Key获取方式：

Reasoning: ...配置。需要检查app.py中是否正确加载了环境变量，并指导用户正确设置高德的API Key。 >

 **Cline wants to execute this command:**

```
type app.py
```

> Command Output

 Checkpoint

 Compare

 Restore

✓ **API Request**

🚧 **API Request**

Reasoning: ...'0.0.0.0'和port=6666，确保应用监听所有接口。同时，确认路由是否正确处理了GET请求。 >

用户报告在运行Flask服务后无法通过浏览器访问localhost:6666。需要排查以下可能原因：

1. 端口监听问题：
 - 检查Flask应用是否确实在指定端口运行
 - 查看是否有其他进程占用6666端口

首先需要补全index.html文件内容，然后检查关键配置和依赖安装

 **Cline wants to edit this file:**

templates/index.html

⊗ Diff Edit Failed

This usually happens when the model uses search patterns that don't match anything in the file. Retrying...

✓ API Request

Reasoning: ...的元素, 如正确的CSS和JS文件路径, 以及正确的HTML结构, 以支持前端功能正常运行。

✎ Cline wants to edit this file:

static/css/style.css

🔖 Checkpoint

Compare

Restore

✓ API Request

Reasoning: ...逻辑, 可能需要询问用户关于大模型的使用方式。5. 测试整个应用, 确保所有功能正常。

✎ Cline wants to edit this file:

static/js/app.js

```
async function getWeather() {
  const city1 = document.getElementById('city1').value.trim();
  const city2 = document.getElementById('city2').value.trim();

  if (!city1 || !city2) {
    alert('请输入两个城市名称');
    return;
  }

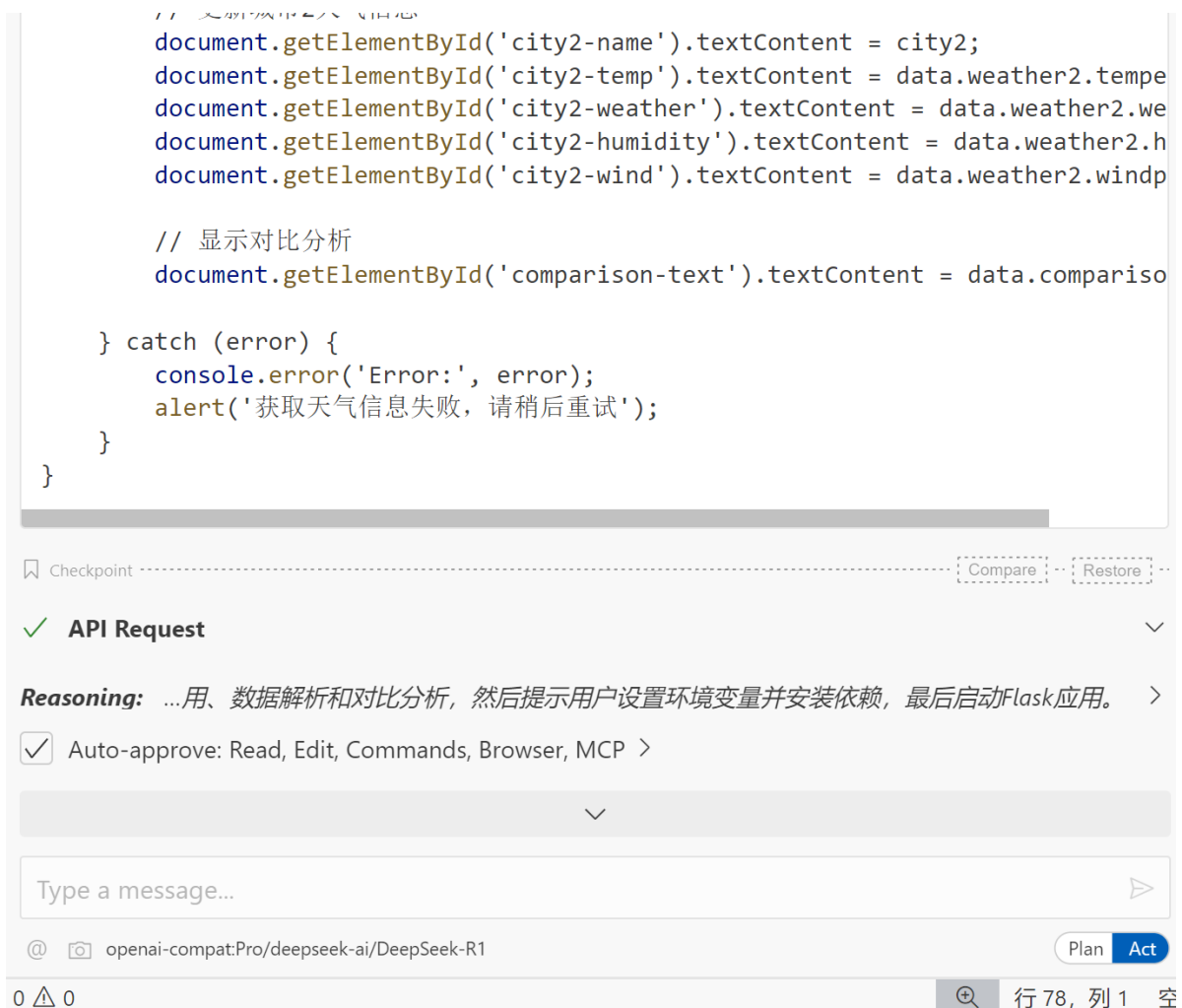
  try {
    const response = await fetch('/compare_weather', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        city1: city1,
        city2: city2
      })
    });

    const data = await response.json();

    if (data.error) {
      alert(data.error);
      return;
    }

    // 更新城市1天气信息
    document.getElementById('city1-name').textContent = city1;
    document.getElementById('city1-temp').textContent = data.weather1.tempe
    document.getElementById('city1-weather').textContent = data.weather1.we
    document.getElementById('city1-humidity').textContent = data.weather1.h
    document.getElementById('city1-wind').textContent = data.weather1.windp

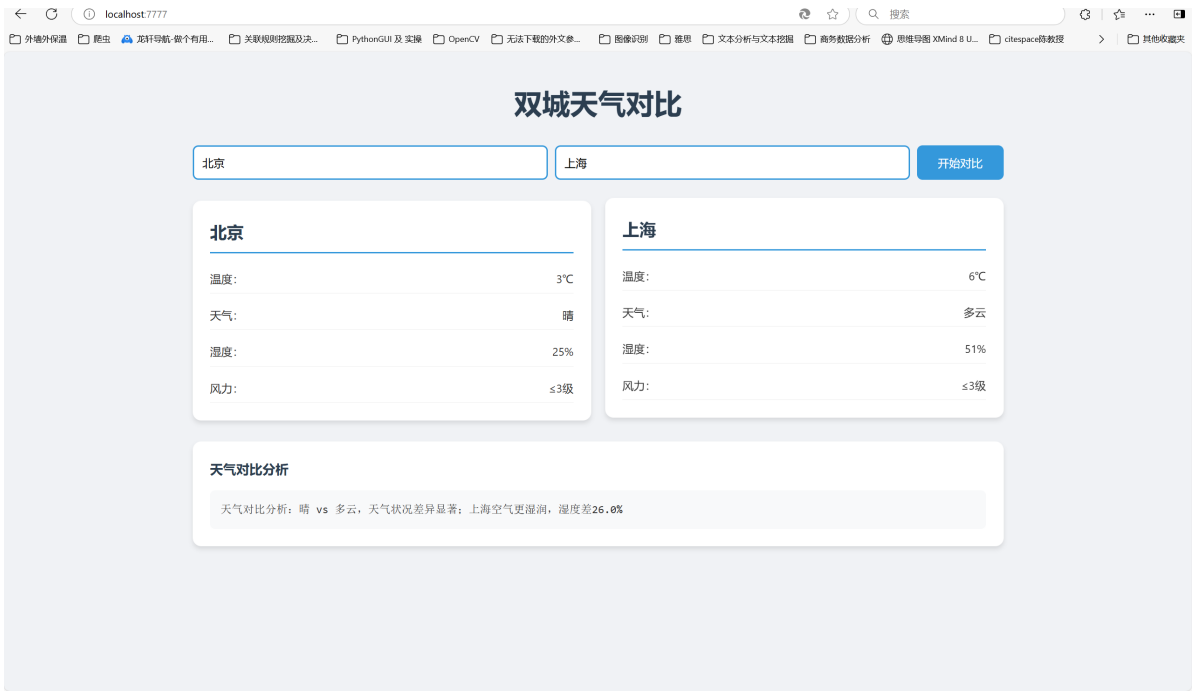
    // 更新城市2天气信息
```



8. 经过多轮交互，最终通过命令行启动服务



9. Web端访问6666端口，输入城市后点击开始对比按钮，效果如下



步骤4：效果如下

